

Literature Review

ZeroTrustCloud: Implementing and Automating Identity-Centric Zero Trust Architecture for Cloud-Native Environments

Submitted as part of Week 3 Milestone

Name: **BAIRE GOWDA**

Reg No: **25PG00157**

1. Introduction

For Week 1 I focused on papers that laid out the overall shape of the problem, mostly the standards and the systematic reviews that told us zero trust is the right direction and roughly what the six components of ZeroTrustCloud should be doing. For this Week 3 submission I went looking for papers that go a level deeper into each of those six components: how identity actually gets federated across trust domains that were never set up ahead of time to talk to each other, how someone would automate RBAC/ABAC as code instead of just recommending it, what it actually costs in latency and CPU to run Istio with mTLS switched on, how bad GitHub Actions misconfigurations really are once you look across hundreds of thousands of real workflow files, and so on.

I again tried to pick papers published mostly in the 2025-2026 window, and made sure I had at least one paper touching each of the six ZeroTrustCloud engines so nothing gets left without literature behind it. I also included one government report this time, CISA's account of how federal agencies have actually been doing at zero trust adoption over the past few years, because it felt useful to weigh the more academic, idealized papers against a report that's honest about how slow and uneven this kind of migration is in practice.

2. Reviewed Literature

1. Zero Trust Architecture Implementation: Report to Congress

Cybersecurity and Infrastructure Security Agency, CISA (2025) | Report submitted under Executive Order 14028 and OMB M-22-09

Link: https://www.dhs.gov/sites/default/files/2025-04/2025_0129_cisa_zero_trust_architecture_implementation.pdf

What They Did: This isn't an academic paper, it's CISA's own account of how U.S. federal civilian agencies (FCEB) have actually been progressing on zero trust since the 2021 executive order. CISA used its own Zero Trust Maturity Model as the yardstick and pulled in data from the Continuous Diagnostics and Mitigation (CDM) Dashboard, which agencies use to report their asset inventories, to see where agencies actually stand pillar by pillar (identity, devices, networks, applications and workloads, data) rather than just where they say they stand on paper.

What They Found: Identity came out as the most mature pillar across agencies, mostly because of long-standing PIV/CAC smart-card authentication that predates the zero trust push entirely, while data protection and application-level security lagged well behind. Most agencies were still sitting somewhere between the Traditional and Initial maturity stages overall rather than Advanced or Optimal, even years into a mandated migration. The report frames the CDM-driven

asset baseline work as one of the more concrete wins, since agencies could not really improve any other pillar without first knowing what they actually had running.

Why It Matters to This Project: This was a useful reality check to read after a week of papers that all describe zero trust in fairly idealized terms. It suggests that even with money, a legal mandate, and years of runway, organizations tend to get identity working reasonably well long before they get data and application security working, which lines up with how ZeroTrustCloud is already sequenced (IAM Engine first, Observability & Compliance Engine last). It also made me think we should be honest in our own final report about which of our six engines we expect to reach a genuinely mature state within the project timeline versus which ones will only reach a basic working prototype.

2. Federating Trust Perimeters: Extending Industry IAM with DLT-Based Governance

Segat, C. (2026) | arXiv preprint 2609.06595, Technische Universität Berlin

Link: <https://arxiv.org/pdf/2609.06595>

What They Did: Where the SPIFFE paper I reviewed last week (paper 6) covered identity federation across clusters that were already planned to trust each other, this paper asks a harder question: what happens when two organizations that never configured anything in advance suddenly need to trust each other's workloads, the way an AI agent from one company might need to call a service belonging to a different company it has never talked to before. The author checked whether SPIRE, Workload Identity Federation, and OpenID Federation 1.0 can support this kind of "unplanned" federation, and concluded SPIRE is the best starting point but is still missing three things: a way to mint scoped tokens from a foreign workload's SPIFFE identity, a way to attest a specific workload rather than an entire trust domain, and a shared, tamper-resistant place to publish trust roots and governance decisions, which the author proposes solving with a distributed ledger.

What They Found: The paper is upfront that it only proposes and analytically justifies this design; it does not implement or benchmark it, and the author lists remote attestation, token exchange, and the distributed-ledger layer itself as the three open engineering problems that would need to be solved before any of this could actually be built and tested.

Why It Matters to This Project: This is a good complement to last week's SPIFFE paper because it names a gap our Service Mesh Security Engine doesn't currently need to solve (we're operating inside clusters we control) but might run into the moment ZeroTrustCloud needed to talk to any identity outside our own trust domain, for example a third-party API or a partner cluster. It's also a useful reminder for our own report: being honest that a design is unimplemented, like this paper is, is a lot more credible than quietly presenting an untested idea as if it were proven, which is something I want to be careful about when we write up whichever parts of our own six engines we don't get fully working.

3. Research on Policy-as-Code for Implementation of Role-based and Attribute-based Access Control

Vakhula, O., Opirskyy, I., Vorobets, P., Bobko, O., Kulinich, O. (2025) | CPITS 2025 Workshop, CEUR Workshop Proceedings Vol. 3991

Link: <https://ceur-ws.org/Vol-3991/paper11.pdf>

What They Did: This paper looks specifically at the problem our Policy-as-Code Engine and IAM Engine are supposed to solve together: turning RBAC and ABAC from something an administrator configures by hand into something defined, tested, and deployed as code. The authors walk through three worked case studies with actual sample policy code: Kubernetes RBAC enforced through OPA as an admission controller, ABAC for a healthcare data-access scenario using HashiCorp Sentinel with attributes like job role, IP range, and time of day, and a cross-cloud ABAC setup combining AWS IAM conditions with OPA as a centralized policy decision point across AWS and Azure.

What They Found: Policy-as-code reduced misconfiguration risk and gave organizations continuous, automated compliance evidence through version-controlled, CI/CD-tested policies, which directly echoes what I read about GitOps-based policy reuse last week. But the paper is also honest about real costs: attribute-heavy ABAC policies introduced noticeable latency during real-time evaluation, initial setup and the learning curve for Rego were non-trivial, and without care, ABAC can develop its own version of "role explosion" called policy or attribute sprawl.

Why It Matters to This Project: This gives us working sample code for exactly the OPA-as-admission-controller pattern our Policy-as-Code Engine is meant to implement, and the Sentinel/ABAC case study is a good model if we ever want our IAM Engine to support attribute-based rather than purely role-based decisions. The latency warning for attribute-heavy ABAC is something I think we should actually benchmark in our own prototype rather than assume away, since this paper flags it as one of the main practical limitations rather than a minor detail.

4. Graph Neural Network–Based Adaptive Threat Detection for Cloud Identity and Access Management Logs

Madireddy, V. T. (2025) | arXiv preprint 2512.10280

Link: <https://arxiv.org/pdf/2512.10280>

What They Did: This paper tackles the exact weakness that last week's systematic review (paper 2) flagged: AI-based monitoring that flags something suspicious without being able to explain why. The author models IAM audit logs (of the kind CloudTrail, Azure AD, or GCP IAM produce) as a dynamic, heterogeneous graph where users, roles, and resources are nodes and access events are edges, then trains a Graph Neural Network with an attention mechanism on top to learn which relationships in that graph look normal versus anomalous, and compares it against Random Forest, XGBoost, and LSTM baselines on the same synthetic and real-world IAM data.

What They Found: The GNN clearly outperformed the baselines, reaching 0.93 precision, 0.91 recall, and a 0.92 F1-score against the LSTM baseline's 0.85 F1, while cutting the false positive rate down to 0.04 compared to 0.07-0.09 for the other models. It also stayed fast, averaging under 35ms of latency even at 20,000 events per second. An ablation study showed the attention

mechanism alone added 6.3% to recall by highlighting which relationships (like a rare privilege escalation) mattered most, and the adaptive retraining loop cut false positives by another 4.8% by continuously updating on confirmed benign and malicious samples.

Why It Matters to This Project: This is a concrete architecture and a concrete set of numbers for the anomaly-detection half of our Continuous Verification Engine, and it directly answers the criticism I flagged from last week's paper 2: because the attention weights show which specific user-role-resource relationship triggered the flag, this isn't a black-box alert, an analyst (or an automated dashboard) can actually see why something looked wrong. I think our own Continuous Verification Engine should aim for something similar, an anomaly score plus the specific relationship that drove it, rather than just a binary suspicious/not-suspicious flag.

5. Autonomous Threat Detection and Response in Cloud Security: A Comprehensive Survey of AI-Driven Strategies

Sarraf, G., Pal, V. (2026) | arXiv preprint 2601.03303; also International Journal of Emerging Research in Engineering and Technology, Vol. 6, Issue 4

Link: <https://arxiv.org/pdf/2601.03303>

What They Did: Rather than building a new model like the GNN paper above, this one is a broad survey of how AI-driven threat detection and response actually gets used across cloud security as a whole, pulling together work on machine learning and deep learning-based intrusion detection, reinforcement learning for adaptive response, and even large language models being used to orchestrate faster, more precise incident response, rather than looking at any one detection technique in isolation the way most single-model papers do.

What They Found: The survey's central argument is that AI's real advantage over older signature- and rule-based detection is its ability to keep learning as threats evolve, which cuts down both false positives and the time it takes to notice something is wrong, and enables things like automatic incident containment and self-healing infrastructure rather than just alerting a human. On the limitations side, it discusses the same data-quality and training-diversity problems noted elsewhere in this week's readings, cloud telemetry varies so much in volume and format across platforms and organizations that normalizing it well enough to train a reliable model is itself a hard problem, separate from designing the model.

Why It Matters to This Project: This is a good reminder that our Continuous Verification Engine and our Observability & Compliance Engine aren't really separate concerns, the anomaly detection piece is only going to be as good as the telemetry pipeline feeding it, which is exactly what the observability engine is responsible for producing. It also makes me want to plan for a normalization/schema step (similar to what the GNN paper above described doing across CloudTrail, Azure AD, and GCP IAM formats) as an explicit part of our design rather than an afterthought, since this survey suggests that's usually where these systems actually run into trouble first.

6. Zero Trust Security Architecture in Microservices-Based Web Applications

IJARCSE Authors (2025) | International Journal of Advanced Research in Computer Science and Engineering, Vol. 1, Issue 3

Link: <https://ijarcse.org/index.php/ijarcse/article/download/68/81>

What They Did: This paper builds and empirically evaluates almost exactly the setup our Service Mesh Security Engine is meant to implement: Istio's service mesh with Envoy sidecars for mTLS and authorization policies, paired with Keycloak as the centralized identity provider issuing JWTs, plus default-deny Kubernetes network policies underneath. They ran a testbed with two realms in Keycloak (one for users, one for services) and measured both normal performance overhead and how the system held up under simulated DoS, lateral movement, and replay attacks.

What They Found: The security gains were substantial: attack success rates dropped from an average of 44% down to below 5% across the attack scenarios they tested. The performance cost was real but manageable, authentication latency rose to about 35ms on average and CPU utilization increased by roughly 8%. Under active attack, latency increased further by 12-18%, but the system recovered to 90% of normal throughput within about two minutes after policies adjusted.

Why It Matters to This Project: This gives us actual numbers to set expectations against for our Service Mesh Security Engine rather than just assuming mTLS and Istio AuthorizationPolicies will be "fast enough." The two-realm Keycloak pattern (separating user identity from service identity) is also a detail I hadn't considered and think we should adopt, since it keeps our IAM Engine's user-facing policies cleanly separate from the service-to-service policies the mesh enforces, rather than mixing the two together in one realm.

7. Establishing Workload Identity for Zero Trust CI/CD: From Secrets to SPIFFE-Based Authentication

Avirneni, S. T. (2025) | arXiv preprint 2504.14760

Link: <https://arxiv.org/pdf/2504.14760>

What They Did: This paper is about a gap in our own Pipeline Security Engine's design that I hadn't really thought through: right now our plan covers artifact signing and policy gates, but it doesn't say anything about how the CI/CD runner itself proves who it is before it ever gets to that point. The author traces the evolution from static long-lived secrets, to OIDC federation (where a cloud provider trusts tokens minted by GitHub Actions or GitLab CI directly), to SPIFFE-issued, runtime-minted identities (SVIDs) for the CI job itself, arguing each step removes another category of standing credential that could leak.

What They Found: Like the federation paper above, this is a position/design paper rather than an implementation study, the author is explicit that no experiments or benchmarks are included, and the contribution is meant as an architectural synthesis rather than a proven system. It does lay out a clear three-stage comparison table (static secrets vs. OIDC federation vs. SPIFFE) and argues SPIFFE is the only one of the three that gives a CI job its own portable, attestable identity rather than borrowing a platform-issued one.

Why It Matters to This Project: This connects our Pipeline Security Engine directly to our IAM/Service Mesh Engine's SPIFFE work from last week, instead of treating pipeline identity as a separate problem, we could issue SPIFFE SVIDs to CI runners the same way we'd issue them to any other workload, and let the same trust infrastructure cover both. Given that this is an unimplemented design paper, though, I think the honest move for our own project is to treat this as an interesting direction worth mentioning in our final report rather than something we commit to actually building, since we'd be building on a pattern nobody has published working numbers for yet.

8. An Empirical Study of Complexity, Heterogeneity, and Compliance of GitHub Actions Workflows

Abrokwah, E., Ghaleb, T. A. (2025) | arXiv preprint 2507.18062, Trent University

Link: <https://arxiv.org/pdf/2507.18062>

What They Did: This is the same kind of large-scale measurement study as last week's CI/CD paper (paper 9), but focused specifically on GitHub Actions workflows rather than CI/CD pipelines in general. The authors pulled 27,863 real workflow files from nearly 7,700 active, non-toy Java, Python, and C++ repositories, then statically analyzed and manually reviewed a stratified sample of 382 of them to measure structural complexity, how consistently workflows follow common step sequences like checkout-setup-test-deploy, and how well they comply with GitHub's own documented best practices around things like version pinning, permission scoping, and dependency caching.

What They Found: Compliance problems turned out to be systematic rather than occasional: secure SHA-based pinning of third-party actions was rarely used even though it is the only pinning method immune to tag-mutation supply-chain attacks (the kind behind the real 2025 tj-actions/changed-files incident), 42.1% of workflows built code without running any tests at all, and reusable/modular workflow features went largely unused despite being supported by the platform. The authors describe these as platform-wide, ecosystem-driven gaps rather than isolated developer mistakes.

Why It Matters to This Project: Where last week's CI/CD measurement paper made the general case that pipeline security is a real, widespread problem, this paper gives a specific, checkable practice, SHA-pinning third-party actions instead of tag-pinning, that our Pipeline Security Engine could realistically implement as an automated check, essentially a starter item rather than an abstract category. The tj-actions incident they cite is also a useful concrete example to put in our final report as evidence that even a basic, working implementation of pinning checks in ZeroTrustCloud would already catch something that almost every real-world workflow currently gets wrong.

9. SBOMproof: Beyond Alleged SBOM Compliance for Supply Chain Security of Container Images

Bufalino, J., Di Francesco, M., Blaise, A., Secci, S. (2025) | arXiv preprint 2510.05798

Link: <https://arxiv.org/pdf/2510.05798>

What They Did: This paper picks up almost exactly where last week's ARGO-SLSA paper (paper 8) left off. That paper assumed a signed, well-formed SBOM was trustworthy evidence of what's inside a container image; this paper actually tests that assumption by comparing several widely used SBOM generation and vulnerability-scanning tools (both open-source tools and major cloud providers' services) against each other on the same Debian and Alpine base images, the two most common Linux distributions used for containers.

What They Found: Different tools disagreed with each other in how they identified the same packages and CVEs, even when scanning the exact same image, because package identifiers in SBOMs aren't consistently interoperable across tools. The authors call this the "SBOM confusion" vulnerability: an image can look compliant and fully documented on paper while still hiding real, undetected vulnerabilities simply because the tool that generated the SBOM and the tool that scans it for CVEs don't agree on how to name the same package. They also built and tested a translation layer between tools that measurably improved interoperability.

Why It Matters to This Project: This is a direct complication for our Pipeline Security Engine's artifact-signing plan, signing an SBOM cryptographically (which is what the ARGO-SLSA paper from last week covers) only proves the SBOM wasn't tampered with after it was generated, it says nothing about whether the SBOM itself is complete or accurate in the first place. If we implement SBOM generation as part of our pipeline, this paper makes the case that we should pick one tool and be consistent about it end to end (generation and scanning) rather than assuming any SBOM is trustworthy just because it's present and signed.

10. Continuous Observability Assurance in Cloud-Native Applications

Borges, M. C., Werner, S. (2025) | arXiv preprint 2503.08552, Technische Universität Berlin

Link: <https://arxiv.org/abs/2503.08552>

What They Did: This paper argues that most teams treat observability instrumentation as a set of ad-hoc decisions (what to log, what to trace, what thresholds to alert on) rather than something that can be systematically designed and tested, and that some of those decisions meaningfully affect how fast a fault actually gets detected. The authors built a tool called OXN that injects faults into a running microservice application (similar in spirit to chaos engineering) while also being able to modify the observability configuration itself, so they can directly measure how well a given instrumentation setup actually detects the faults it's supposed to catch, rather than just assuming it does.

What They Found: By treating fault detection as a testable, quantifiable property instead of a one-time setup decision, the method lets a team run repeatable observability experiments and compare configurations against each other on actual detection effectiveness and cost, rather than relying on intuition about which metrics or traces "should" matter. The authors present this as a preliminary demonstration and are explicit that further validation with real SRE teams is still needed.

Why It Matters to This Project: This connects back to last week's governance architecture paper (paper 10), which argued that observability data and compliance evidence are really the same thing, not two separate features. This paper adds a piece that one was missing: a way to actually test whether our Observability & Compliance Engine's instrumentation is good enough to catch the faults and anomalies it's supposed to, rather than just assuming that collecting logs,

metrics, and traces automatically means we have adequate coverage. I think it would strengthen our final evaluation if we could show, even in a small way, that our observability setup actually detects injected faults rather than just displaying a dashboard that looks complete.

3. Overall Takeaway

Where the Week 1 papers mostly justified why ZeroTrustCloud's six-component design makes sense, this week's papers pushed me to think harder about the seams between those components rather than the components themselves. The CISA report and the two design-only papers (federation and CI/CD workload identity) were useful precisely because they were honest about limits, either how far real agencies actually get in practice, or how much of a proposed architecture is still just a proposal. The empirical papers (Istio/Keycloak, GitHub Actions misconfigurations, SBOM tool disagreement, and the GNN detector) gave me actual numbers I can use to set realistic expectations for our own prototype instead of just assuming things like mTLS or SBOM signing "work" once they're switched on.

The clearest connective thread across the ten papers is that no single engine in ZeroTrustCloud is really self-contained: the Continuous Verification Engine's anomaly detection is only as good as the Observability Engine's telemetry, the Pipeline Security Engine's artifact signing is only as trustworthy as the SBOM tooling feeding it, and the Service Mesh Security Engine's identity model could plausibly extend all the way down into the CI/CD runners that build the workloads in the first place. If Week 1 was about proving each engine belongs in the project, Week 2 has mostly convinced me that the integration between engines is where our own implementation is going to be tested hardest.